MAML-Select: An Online Adaptive Client Selection Method for Federated Learning via Meta-Learning

***Abstract*—In Federated Learning (FL), choosing which clients update the shared model in each round is a combinatorial scheduling problem under resource constraints. Clients differ in data utility, training speed, battery capacity and bandwidth. Since only a small cohort can communicate per round, conventional strategies favor high-utility or fast clients, which repeatedly selects a narrow group, leaves slower ones underused, and reduces participation coverage. Existing selectors rely on a fixed scoring rule, or update it slowly, even though a client's usefulness changes as the global model evolves. Model-Agnostic Meta-Learning Select (MAML-Select) removes this assumption and frames each round as a fast adaptation task. A lightweight policy network is adapted from the previous round's observed loss reduction and latency and used to rank clients. After the selected clients return their updates, the same policy is meta-updated using the new feedback. We evaluate MAML-Select on Fashion-MNIST, CIFAR-10, and CIFAR-100 with 100 clients, K = 10 selected clients per round, a Dirichlet non-independent and identically distributed (non-IID) split at $\alpha = 0.5$, and three seeds. Against FedAvg on the three datasets in turn, MAML-Select lowers cumulative Tera Floating-Point Operations (TFLOPs) by 12.9%, 21.5% and 1.7% and modelled energy by 16.4%, 24.8% and 4.7%, for accuracy changes of -0.1, -3.8 and -1.5 percentage points, and it lowers the mean CIFAR-100 round latency by 14.4%. The saving is small on CIFAR-100, where FedGCS leads on accuracy. Participation coverage stays at 100% on every dataset, through a single exploration slot.**

***Impact Statement*—FL is central to privacy-preserving AI across heterogeneous environments, such as mobile health, intelligent transportation, and smart-city systems, but these face tight computational-capacity, energy-consumption, and latency budgets. MAML-Select contributes an adaptive selection mechanism that reduces client-training computational cost while preserving broad participation coverage. By using recent training feedback to re-rank clients, it supports FL systems that respond to changing device conditions instead of relying on a fixed rule. The reported energy values are modelled estimates that follow the declared simulator and device-tier setup, giving a transparent basis for comparing selection policies. The broader impact is a more resource-aware path for deploying distributed AI at the edge, including the industrial Internet of Things (IoT).**

***Index Terms*—Federated learning, client selection, meta-learning, computational efficiency.**

I. Introduction

FEDERATED Learning (FL) trains a shared model across many devices without moving their raw data to the server [1]. Clients train locally and share only model updates, but differ in hardware, network conditions, and data. This creates drift under non-independent and identically distributed (non-IID) data and straggler delays in synchronous training [2]–[6]. The problem of *whom to select* each round remains open.

The implementation is available at https://anonymous.4open.science/r/MAML-Select-REPLACE-BEFORE-SUBMIT.

Client selection chooses a subset $\mathcal{S}_t \subset \{1, \ldots, N\}$ each round. Client utility is non-stationary, because a useful client can become redundant later, while an overlooked client can become important in the long run. System-aware methods improve participation but can leave slow yet data-rich clients underused [7], [8], and utility-aware methods prioritize information gain at the cost of latency or expensive server-side modelling [9]–[13]. Learning-based selectors use histories, features, or reinforcement signals [14]–[16]. A round-by-round mechanism for adapting the policy to evolving utility remains underexplored.

We propose MAML-Select, a client-selection framework based on a Model-Agnostic Meta-Learning (MAML)-style fast adaptation step [17], which learns the policy network via recent feedback before the next ranking decision. Unlike reinforcement-learning (RL)-based or contextual-bandit selectors, the proposed MAML-Select uses rapid gradient-based policy adaptation. Unlike personalization of FL client models [18], the proposed MAML adapts the selector, rather than the client model. The approach suits devices with limited computational capacity, strict deadlines, or intermittent connectivity. The contributions are as follows.

1) We formulate the FL client selection problem as a meta-learning task and introduce the MAML-Select algorithm with rapid gradient-based adaptation for online client selection.
2) Experiments on Fashion-MNIST, CIFAR-10, and CIFAR-100 under non-IID settings ($\alpha = 0.5$, three seeds) show that MAML-Select delivers accuracy competitive with FedAvg and FedGCS at lower computational cost, and higher accuracy than FedCS, Oort, TiFL, and FedCor. Sensitivity and ablation studies over λ, the inner-loop steps, the selector width, and the state features support the design choices.
3) We demonstrate that MAML-Select *lowers cumulative training TFLOPs* by up to 21.5% and modelled energy consumption by up to 24.7% relative to FedAvg, with paired-test support for the resource reductions.

Organization. The rest of this paper is organized as follows. Section II reviews related work on client selection in FL. Section III details the system model and problem formulation. The proposed MAML-Select algorithm is described in Section IV, followed by evaluation and results in Section V. Section VI concludes this paper and discusses future research directions.

II. Related Work

A. System-Aware Selection

System-aware selection addresses stragglers. FedCS selects feasible clients under resource constraints, while TiFL groups

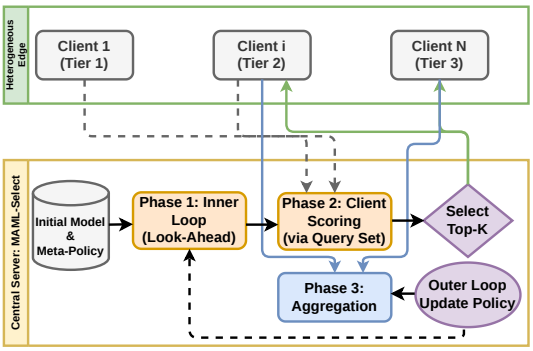

Fig. 1. System model and the per-round workflow of MAML-Select. The server holds the global model and the ranking policy h_ϕ. At the start of round t it adapts the policy on the support set and meta-updates it on the query set, both drawn from completed rounds, then scores the available clients and broadcasts to the selected cohort $\mathcal{S}_t$. The cohort trains locally and returns updates with the loss and resource feedback that forms the next round's sets.

devices into online performance tiers [7], [8]. Recent studies show similar concerns for slow but data rich clients in resource-constrained medical IoT and asynchronous FL with heterogeneous objectives [4], [5]. MAML-Select uses feedback from the previous training phase, and learns the selection policy from six client-state features.

B. Data-Aware and Utility-Based Selection

Data-aware methods select clients for model utility rather than speed alone. Oort prioritizes data-rich and fast clients, and FedCor models correlations between client losses [9], [10]. CriticalFL uses critical learning periods, while FedGCS searches for client combinations in a learned continuous space [12], [13]. Other work couples selection with aggregation design [11], [19]. MAML-Select instead targets rapid adaptation of the ranking policy as client utility evolves.

C. Learning-Based and Meta-Learning Approaches

Learning-based selectors improve from observed feedback. FAVOR uses a deep Q-network, while neural contextual bandits learn from client features and rewards [14], [15], and recent work studies deep-RL selection for robustness and fairness-aware participation [16], [20]. MAML is used in [18] to personalize client models, whereas MAML-Select adapts the server-side ranking policy instead.

Table I consolidates these distinctions. For methods without a stated closed-form selection complexity, it reports the dominant server-side operation.

Difference from prior work. MAML-Select meta-learns the selection policy h_ϕ. Unlike heuristic, RL, contextual-bandit, and generative selectors, it uses a MAML inner update to convert recent feedback into fast policy weights before the next cohort is scored. A fixed heuristic or bandit may update rewards over time, but does not adapt the scoring-network parameters from a support set immediately before selection.

III. System Model and Problem Formulation

We consider a central server and N heterogeneous clients indexed by $\mathcal{N} = \{1, 2, \ldots, N\}$. Client i stores a local dataset $\mathcal{D}_i$ with n_i samples. The global objective is to minimize the empirical risk over the distributed data as

$$\min_{\theta \in \mathbb{R}^d} F(\theta) = \sum_{i=1}^{N} \frac{n_i}{n} F_i(\theta), \tag{1}$$

where $\theta \in \mathbb{R}^d$ denotes the global model, $n = \sum_{i=1}^{N} n_i$, and $F_i(\theta) = \frac{1}{n_i} \sum_{j=1}^{n_i} \ell(\theta; x_{i,j}, y_{i,j})$ is the local empirical risk, with $\ell(\cdot)$ the per-sample loss (e.g. cross-entropy).

Fig. 1 shows the setting and the order of operations in one round. At round t, let $\mathcal{A}_t \subseteq \mathcal{N}$ be the available clients and $\mathcal{S}_t \subseteq \mathcal{A}_t$ the selected cohort. The server holds the global model and the policy, while clients keep data local and return only updates, loss summaries, and resource-state feedback, so selection is a server-side decision taken before each broadcast.

Client i has per-round latency $T_{i,t} = T_{i,t}^{comp} + T_{i,t}^{comm}$. The computation time is $T_{i,t}^{comp} = E\, n_i / f_{i,t}$ over E local epochs and n_i samples, where $f_{i,t}$ is the device throughput in samples per second, set by its hardware tier. The per-sample model cost is absorbed into it. The communication time is $T_{i,t}^{comm} = n_i/(\kappa B_{i,t})$ for a payload scale κ and uplink quality $B_{i,t}$ [7], and it stays under 1% of $T_{i,t}^{comp}$ here, so local training dominates. In synchronous FL the slowest selected client sets the latency,

$$T_t^{round} = \max_{i \in \mathcal{S}_t} T_{i,t}. \tag{2}$$

Let $a_{i,t}$ indicate whether client i is selected. Under policy π_ϕ, the constrained client-selection problem is

$$\begin{aligned} \min_{\pi_\phi} \quad & \sum_{t=1}^{T} \left[F(\theta_{t+1}) + \lambda T_t^{round} \right] \\ \text{s.t.} \quad & \sum_{i=1}^{N} a_{i,t} = K, \quad a_{i,t} \in \{0,1\}, \\ & a_{i,t} \le \mathbf{1}\{i \in \mathcal{A}_t\}, \end{aligned} \tag{3}$$

where K is the cohort size, T is the number of rounds, and $\lambda \ge 0$ controls the trade-off between latency and loss, and the client selection subset is updated as $\mathcal{S}_t = \{i : a_{i,t} = 1\}$. A larger λ places more emphasis on latency, while $\lambda = 0$ yields loss-only selection. We assume $|\mathcal{A}_t| \ge K$. The model θ_{t+1} results from local training and aggregation over $\mathcal{S}_t$. The problem remains challenging because the action space is combinatorial and client utility evolves during training.

IV. MAML-Select Algorithm

Building on (3), MAML-Select treats selection as a meta-learning problem, since client utility changes during training.

A. State Space

The server builds a state vector $\boldsymbol{s}_{i,t} \in \mathbb{R}^6$ for each client i at round t. It holds the local training loss $\hat{L}_{i,t}$, the gradient norm $\|\nabla F_i\|_2$, the estimated latency $\hat{T}_{i,t}$, the battery level $b_{i,t}$, the selection frequency $\nu_{i,t}$, and the staleness $\tau_{i,t}$ measured in rounds since the client was last selected [11], [15]. The three pairs capture statistical utility, system feasibility, and participation balance. Each round t is a task $\mathcal{T}_t$ in which clients are ranked so the top-K cohort minimizes the per-round cost.

TABLE I
COMPARISON OF REPRESENTATIVE METHODS AND MAML-SELECT.

Method	Primary objective	Adaptation mechanism	Selection complexity or dominant operation
FedCS [7]	Increase feasible updates under resource constraints	Resource estimates before selection	Resource-constrained subset search
TiFL [8]	Mitigate stragglers while preserving accuracy	Online tier updates	Tiering and probabilistic sampling
Oort [9]	Improve time-to-accuracy	Utility and speed feedback	Utility scoring and guided sampling
FedCor [10]	Accelerate convergence under non-IID data	Loss-correlation model	Gaussian process (GP) covariance operations
FAVOR [14]	Counterbalance non-IID bias	Experience-driven policy updates	DQN inference and training
NCCB [15]	Learn contextual client combinations	Client features and reward feedback	Locality-sensitive hashing (LSH) feature extraction and bandit updates
FedCG [11]	Couple selection and gradient compression	Capability-aware compression updates	Submodular optimization and linear programming
CriticalFL [12]	Exploit critical learning periods	Period-aware participation	Base-selector dependent
FedGCS [13]	Optimize client combinations	Search in a learned continuous space	Latent-space optimization and beam search
Kazemi et al. [16]	Improve robustness under poisoning attacks	Experience-driven policy updates	Deep-RL inference and training
Vox Populi [20]	Promote fair and inclusive participation	Fairness-aware client selection	Framework-dependent
Per-FedAvg [18]	Personalize the client model to local data	MAML adapts the client model, not the selector	No client selection, only per-client fine-tuning
MAML-Select	Adapt rankings under evolving client utility	Fast adaptation from recent round feedback	$\mathcal{O}(N\|\phi\|)$ scoring plus a gradient-adaptation step

The six raw entries carry different physical units. Before they reach the policy, each column is therefore standardized across the clients available in that round,

$$\boldsymbol{s}_{i,t} \leftarrow \left(\boldsymbol{s}_{i,t}^{raw} - \boldsymbol{\mu}_t\right) \oslash \left(\boldsymbol{\sigma}_t + \epsilon_0\right), \tag{4}$$

where $\boldsymbol{\mu}_t, \boldsymbol{\sigma}_t$ are the per-feature mean and standard deviation over $\mathcal{A}_t$, $\oslash$ is elementwise division, and $\epsilon_0 = 10^{-8}$. The policy input is then dimensionless and the score is a relative ranking within one round, which is all Top-K needs.

MAML-Select trains a meta-policy network $h_\phi(\boldsymbol{s}_{i,t})$, parameterized by ϕ, which is adapted to the current round using recent feedback. The policy is a 6-64-64-1 multilayer perceptron (MLP) with rectified linear unit (ReLU) activations and 4,673 trainable parameters. Cost feedback for a cohort becomes observable only after that cohort has finished training, so the two meta-learning sets are drawn from two consecutive completed rounds. Writing $\mathcal{S}_t$ for the cohort of round t, the support and query sets used at round t are

$$\begin{aligned} \mathcal{D}_{sup}(t) &= \left\{(\boldsymbol{s}_{i,t-2}, c_{i,t-2})\right\}_{i\in\mathcal{S}_{t-2}}, \\ \mathcal{D}_{query}(t) &= \left\{(\boldsymbol{s}_{i,t-1}, c_{i,t-1})\right\}_{i\in\mathcal{S}_{t-1}}, \end{aligned} \tag{5}$$

and therefore $\mathcal{D}_{sup}(t) = \mathcal{D}_{query}(t-1)$. This lag is what makes the procedure a meta-learning problem rather than online regression, because the inner step adapts on an older round while the meta-gradient is measured on a newer one. The outer update therefore rewards initializations from which one adaptation step transfers forward in time, which is what a selector needs when utility drifts. The candidate set $\mathcal{D}_{cand}(t) = \{\boldsymbol{s}_{i,t} \mid i \in \mathcal{A}_t\}$ carries no labels and is only scored.

B. From the Cohort Objective to a Per-Client Score

The objective in (3) is defined over cohorts, whereas the selector ranks clients individually. We connect the two in two steps, and only the second is inexact.

First, the straggler term is a maximum and is therefore not separable. For any deadline $T_{target} > 0$ and any cohort $\mathcal{S}_t$,

$$\frac{T_t^{round}}{T_{target}} \;\le\; 1 + \sum_{i\in\mathcal{S}_t} \rho_{i,t}, \qquad \rho_{i,t} \;=\; \frac{\left(T_{i,t} - T_{target}\right)_+}{T_{target}}, \tag{6}$$

where $(x)_+ = \max(0, x)$. The left side is at most 1 when every selected client meets the deadline, and otherwise the slowest client j contributes $1 + \rho_{j,t} \le 1 + \sum_i \rho_{i,t}$. The bound never understates the latency cost, makes it additive, and renders the penalty dimensionless so it can be traded against a loss reduction through one unitless weight λ.

Second, the risk decrease $F(\theta_t) - F(\theta_{t+1})$ is unknown when the cohort is chosen, so we replace it by $\sum_{i\in\mathcal{S}_t} \Delta_{i,t}$, the observable local loss reductions $\Delta_{i,t} = \hat{L}_{i,t}^{before} - \hat{L}_{i,t}^{after}$ over the E local epochs. This is the one approximation in the derivation. It treats cohort contributions as additive and equates local with global progress, ignoring client drift and interference between updates, and we adopt it because it needs no validation set and no extra communication.

Substituting (6) and this surrogate into the per-round term of (3) and dropping the additive constant λ gives a cohort surrogate that is modular in $\mathcal{S}_t$,

$$\widetilde{F}_t(\mathcal{S}) = \sum_{i\in\mathcal{S}} c_{i,t}, \qquad c_{i,t} = \lambda\,\rho_{i,t} \;-\; \Delta_{i,t}, \tag{7}$$

which is the per-client cost used throughout this work and the quantity the policy is trained to predict. Both terms are dimensionless, since $\rho_{i,t}$ is a fractional deadline overrun and $\Delta_{i,t}$ a loss difference. The deadline T_{target} is the mean expected round latency of the Tier-2 devices, recomputed every round, with the pool median used if none is available.

Proposition 1 (Top-K is exact for the surrogate)**.** *Let $\mathcal{S}^\star$ hold K indices of $\mathcal{A}_t$ with the smallest $c_{i,t}$. Then $\mathcal{S}^\star$ minimizes $\widetilde{F}_t$ over all subsets of $\mathcal{A}_t$ of size K.*

Proof. Suppose some feasible $\mathcal{S}$ has a smaller objective. Since both sets have size K, there are $u \in \mathcal{S}\setminus\mathcal{S}^\star$ and $v \in \mathcal{S}^\star\setminus\mathcal{S}$ with $c_{v,t} \le c_{u,t}$. Swapping u for v does not increase the objective and strictly reduces $|\mathcal{S} \setminus \mathcal{S}^\star|$, so at most K swaps turn $\mathcal{S}$ into $\mathcal{S}^\star$ without ever increasing it, a contradiction.

The chain is therefore an exact bound, one stated approximation, and an exact combinatorial step. One gap remains, since $c_{i,t}$ depends on quantities realized only after client i has trained, and the meta-policy fills it by predicting $c_{i,t}$, after which Proposition 1 applies to the predicted costs. The cohort therefore exactly optimizes a stated modular surrogate under predicted costs.

C. The MAML-Select Algorithm

The algorithm proceeds in three phases per round, all executed at the start of round t in the order below. Both meta-learning updates therefore complete before any client is scored,

and they use only the lagged feedback of (5), so no quantity from round t selects the cohort of round t.

Phase 1, inner loop adaptation. At the start of round t the server adapts ϕ_t on $\mathcal{D}_{sup}(t)$ with one gradient step,

$$\phi_t' = \phi_t - \beta \nabla_{\phi_t} \mathcal{L}_{sup}(h_{\phi_t}; \mathcal{D}_{sup}(t)), \tag{8}$$

where $\mathcal{L}_{sup}$ is the mean-squared-error (MSE) loss between predicted scores and observed costs from round $t-2$, and β is the inner learning rate. The step is plain gradient descent with $\beta = 0.01$ and a single inner step.

Lemma 1 (Inner-step descent)**.** *Let the support loss $g_t(\phi) = \mathcal{L}_{sup}(h_\phi; \mathcal{D}_{sup}(t))$ be L-smooth, that is, ∇g_t is L-Lipschitz. If the inner step size obeys $0 < \beta < 2/L$, the inner update* (8) *is a descent step on the support loss,*

$$g_t(\phi_t') \le g_t(\phi_t) - \beta\Big(1 - \tfrac{L\beta}{2}\Big)\|\nabla g_t(\phi_t)\|_2^2.$$

Proof. By L-smoothness, g_t obeys the descent lemma

$$g_t(\psi) \le g_t(\phi) + \nabla g_t(\phi)^\top(\psi - \phi) + \tfrac{L}{2}\|\psi - \phi\|_2^2.$$

Substituting $\psi = \phi_t'$, $\phi = \phi_t$, and the inner update $\phi_t' - \phi_t = -\beta \nabla g_t(\phi_t)$ yields the stated bound. Its coefficient $\beta(1 - L\beta/2)$ is positive precisely for $0 < \beta < 2/L$, so the step never increases g_t and strictly decreases it unless $\nabla g_t(\phi_t) = 0$.

A single support-set step therefore reduces the ranking loss, which justifies adapting before the candidates are scored.

Phase 2, outer loop meta-update. The meta-policy is updated on the query set of (5),

$$\phi_{t+1} \leftarrow \phi_t - \eta \nabla_{\phi_t'} \mathcal{L}_{query}(h_{\phi_t'}; \mathcal{D}_{query}(t)), \tag{9}$$

where η is the meta-learning rate. Query-loss gradient is evaluated at the adapted weights ϕ_t' and applied to the meta-parameters ϕ_t, which omits the second-order derivatives. Because $\mathcal{D}_{query}(t)$ is the feedback of round $t-1$ and $\mathcal{D}_{sup}(t)$ that of round $t-2$, this update measures how well one adaptation step on older feedback predicts the round that followed it. In the implementation the direction $\nabla_{\phi_t'} \mathcal{L}_{query}$ is applied through Adam at $\eta = 10^{-3}$ rather than by plain descent, and Remark 1 states what this means for the analysis.

Phase 3, selection and FL execution. The adapted $h_{\phi_t'}$ scores $\mathcal{A}_t$ and takes the K lowest predicted costs,

$$\mathcal{S}_t = \text{Top-}K_{i \in \mathcal{A}_t}\left(-h_{\phi_t'}(\boldsymbol{s}_{i,t})\right). \tag{10}$$

Two rules qualify (10). During the first $\lceil N/K \rceil$ rounds the policy has too little feedback to rank anything, so a cold start takes the cohort in a fixed seed-shuffled coverage order that visits every client once. Thereafter an exploration slot gives one of the K places to the available client of largest staleness $\tau_{i,t}$, ties broken by the smallest selection frequency $\nu_{i,t}$ and then by the coverage order, and the rest to the Top-$(K-1)$ of the learned ranking. The slot is deterministic, is set to $\varepsilon = 1$, and bounds how long any client can stay unobserved. Sec. V-C verifies that coverage does not depend on the cold start.

The selected clients then train, and their states and realized costs become the query set of round $t+1$ and the support set of round $t+2$. Algorithm 1 gives the full procedure.

Lemma 2 (Outer-step descent)**.** *Let the query loss $q_t(\phi) = \mathcal{L}_{query}(h_\phi; \mathcal{D}_{query}(t))$ be L_q-smooth, and let*

$$\delta_t = \|\nabla q_t(\phi_t') - \nabla q_t(\phi_t)\|_2$$

denote the look-ahead displacement, the gap between the direction actually applied, $\nabla q_t(\phi_t')$ at the adapted weights, and the query gradient $\nabla q_t(\phi_t)$ at the un-adapted weights. If $0 < \eta \le 1/L_q$, the meta-update (9) *satisfies*

$$q_t(\phi_{t+1}) \le q_t(\phi_t) - \tfrac{\eta}{2}\|\nabla q_t(\phi_t)\|_2^2 + \tfrac{\eta}{2}\delta_t^2,$$

with the inner step controlling it through $\delta_t \le L_q \beta \|\nabla g_t(\phi_t)\|_2$.

Proof. Write $\phi_{t+1} = \phi_t - \eta\, d_t$ with $d_t := \nabla q_t(\phi_t')$. With $\eta \le 1/L_q$, the descent lemma for q_t gives

$$q_t(\phi_{t+1}) \le q_t(\phi_t) - \eta\, \nabla q_t(\phi_t)^\top d_t + \tfrac{\eta}{2}\|d_t\|_2^2.$$

Completing the square with $-a^\top b + \frac{1}{2}\|b\|_2^2 = \frac{1}{2}\|b-a\|_2^2 - \frac{1}{2}\|a\|_2^2$, for $a = \nabla q_t(\phi_t)$ and $b = d_t$, gives

$$q_t(\phi_t) - \tfrac{\eta}{2}\|\nabla q_t(\phi_t)\|_2^2 + \tfrac{\eta}{2}\|d_t - \nabla q_t(\phi_t)\|_2^2.$$

Since $\|d_t - \nabla q_t(\phi_t)\|_2 = \delta_t$, the first bound follows. The second uses L_q-smoothness and (8), which give $\delta_t \le L_q\|\phi_t' - \phi_t\|_2 = L_q\beta\|\nabla g_t(\phi_t)\|_2$.

Since the bound makes δ_t proportional to β for a bounded support-gradient norm, the error term $\frac{\eta}{2}\delta_t^2 = \mathcal{O}(\beta^2)$ is small at $\beta = 10^{-2}$, and the first-order meta-update behaves like gradient descent on the query loss up to that $\mathcal{O}(\beta^2)$ floor.

Two clarifications follow. First, δ_t measures how far the applied direction sits from the query gradient at ϕ_t, which is what the descent inequality needs, and Lemma 2 controls that displacement. Second, a rate follows only if the query objective is fixed across rounds, which is untenable here. Since $\mathcal{D}_{sup}(t) = \mathcal{D}_{query}(t-1)$, setting $q_t \equiv q$ would force $g_t \equiv q$ and collapse the problem into descent on a fixed loss, and it would assert that client utility never changes, which Sec. V-D shows is false. We therefore state a tracking bound.

Corollary 1 (Tracking under a drifting query objective)**.** *Let Lemma 2 hold at every round with $0 < \eta \le 1/L_q$, let $q_t \ge q_{\min}$ for every t, and let $V_T = \sum_{t=1}^{T}(q_{t+1}(\phi_{t+1}) - q_t(\phi_{t+1}))$ be the cumulative drift, measured at a common iterate. Then*

$$\min_{1 \le t \le T}\|\nabla q_t(\phi_t)\|_2^2 \;\le\; \frac{2\big(q_1(\phi_1) - q_{\min}\big)}{\eta T} + \frac{2V_T}{\eta T} + \frac{1}{T}\sum_{t=1}^{T}\delta_t^2.$$

Proof. Lemma 2 gives $\frac{\eta}{2}\|\nabla q_t(\phi_t)\|_2^2 \le q_t(\phi_t) - q_t(\phi_{t+1}) + \frac{\eta}{2}\delta_t^2$. Consecutive terms belong to different objectives, so write $q_t(\phi_t) - q_t(\phi_{t+1}) = [q_t(\phi_t) - q_{t+1}(\phi_{t+1})] + [q_{t+1}(\phi_{t+1}) - q_t(\phi_{t+1})]$. Summing, the first bracket telescopes to at most $q_1(\phi_1) - q_{\min}$ and the second to V_T. Dividing by $\eta T/2$ and bounding the average below by its minimum gives the claim.

Remark 1 (Scope of Corollary 1). If the environment settles, so that $V_T = o(T)$, the policy reaches an $\mathcal{O}(\beta^2)$-approximate stationary point at rate $\mathcal{O}(1/(\eta T))$. A fixed objective gives $V_T = 0$, and any $V_T \le 0$ recovers the same rate. If utility drifts at a constant rate the drift term does not vanish and the guarantee degrades to tracking, meaning the policy stays within a drift-proportional neighbourhood of stationarity. The analysis also assumes an outer step $\phi_{t+1} = \phi_t - \eta \nabla q_t(\phi_t')$, whereas the implementation passes that direction through Adam, so it characterizes the applied direction but not Adam's per-coordinate rescaling.

Algorithm 1 MAML-Select Algorithm

Require: clients $\mathcal{N}$ with $|\mathcal{N}| = N$, rounds T, cohort size K, inner LR β, outer LR η, trade-off λ, exploration slot ε

Initialize global model θ_0, meta-policy ϕ_1 and its Adam state, coverage order $\pi \leftarrow$ seeded shuffle of $\mathcal{N}$, and $\mathcal{D}_{sup}, \mathcal{D}_{query} \leftarrow \emptyset$.
for round $t = 1, \ldots, T$ **do**
  Broadcast θ_t; collect states $\boldsymbol{s}_{i,t}$ from $\mathcal{A}_t$, standardized by (4)
  if $\mathcal{D}_{query} \neq \emptyset$ **then**
    $\phi'_t \leftarrow$ inner step (8) on $\mathcal{D}_{sup}$ ▷ Phase 1
    $\phi_{t+1} \leftarrow$ Adam step on ϕ_t along $\nabla_{\phi'_t} \mathcal{L}_{query}(h_{\phi'_t}; \mathcal{D}_{query})$ ▷ Phase 2
  else
    $\phi'_t \leftarrow \phi_t$, $\phi_{t+1} \leftarrow \phi_t$ ▷ no feedback yet
  end if
  score$_i \leftarrow h_{\phi'_t}(\boldsymbol{s}_{i,t})$ for all $i \in \mathcal{A}_t$ ▷ Phase 3
  if $t \leq \lceil N/K \rceil$ **then** ▷ cold start
    $\mathcal{S}_t \leftarrow$ next K clients of π, cyclically.
  else
    $\mathcal{E}_t \leftarrow \varepsilon$ clients of $\mathcal{A}_t$ smallest in $(-\tau_{i,t}, \nu_{i,t})$. ▷ exploration
    $\mathcal{S}_t \leftarrow \mathcal{E}_t \cup$ Top-$(K - \varepsilon)$ of $-$score over $\mathcal{A}_t \setminus \mathcal{E}_t$.
  end if
  Receive $\{\Delta\theta_{i,t}\}_{i \in \mathcal{S}_t}$, local losses, and system metrics
  $\theta_{t+1} \leftarrow \theta_t + \sum_{i \in \mathcal{S}_t} \frac{n_i}{\sum_{j \in \mathcal{S}_t} n_j} \Delta\theta_{i,t}$
  $T_{target} \leftarrow$ mean Tier-2 latency over $\mathcal{A}_t$; costs $c_{i,t}$ by (7), $i \in \mathcal{S}_t$
  $\mathcal{D}_{sup} \leftarrow \mathcal{D}_{query}$, $\mathcal{D}_{query} \leftarrow \{(\boldsymbol{s}_{i,t}, c_{i,t}) \mid i \in \mathcal{S}_t\}$.
end for
return θ_{T+1}

D. Computational Complexity

Let $P = |\phi|$ be the number of meta-policy parameters, so $P = 4{,}673$ here. Scoring N clients costs $\mathcal{O}(NP)$, a size-K heap selects the best K in $\mathcal{O}(N \log K)$, and the inner and outer updates each touch at most the cohort and cost $\mathcal{O}(KP)$. The per-round server-side selection cost is therefore

$$\mathcal{O}(NP + N \log K + 2KP). \tag{11}$$

To validate (11) we measured the selection overhead over 10 rounds for $N \in \{20, 40, 80, 100, 200, 500, 1000\}$ with the selection rate held at 10%. Fig. 3(b) shows it staying between about 20 and 26 ms per round as N grows fifty-fold, while the operation count in Fig. 3(c) grows about linearly. The measured time stays flat because at these pool sizes the small $\mathcal{O}(NP)$ scoring cost is dominated by fixed per-round bookkeeping. Since P and K are fixed in the protocol, MAML-Select scales linearly with N. The memory cost is $\mathcal{O}(P + N + K)$. Correlation-matrix and Gaussian-Process selectors grow beyond linearly in N, and cubically in common FedCor-style implementations [10].

V. Evaluation and Results

We evaluate on three benchmarks. Fashion-MNIST uses LightCNN-9 [21], and CIFAR-10 and CIFAR-100 use ResNet18 [22]. We simulate $N = 100$ clients under a Dirichlet non-IID partition at $\alpha = 0.5$, selecting $K = 10$ clients per round for $T = 200$ rounds, or 150 rounds on CIFAR-100, with $E = 5$ local epochs and batch size 32. Table II reports mean $\pm$ standard deviation over seeds 42, 123, and 2026. Local models use PyTorch default initialization and stochastic gradient descent (SGD) with learning rate 0.01, momentum 0.9, weight decay 10^{-4}, and a constant scheduler. The meta-policy uses one inner step with $\beta = 0.01$ and Adam for the outer update with $\eta = 0.001$. The selector seed is fixed to 2026, so the reported variance reflects training and partition randomness alone, and Algorithm 1 with these parameters fully specifies the simulation. Processing rates follow three device tiers reflecting the systems heterogeneity common in federated deployments [23], namely Tier 1 at 20% and 1.0×, Tier 2 at 50% and 2.0×, and Tier 3 at 30% and 4.0×, so Tier 1 is the slowest tier and Tier 3 the fastest. The same tiers drive resource accounting, with 4 W and 18 Wh batteries for Tier 1, 7 W and 30 Wh for Tier 2, and 12 W and 48 Wh for Tier 3. The deadline T_{target} is the average round latency of Tier 2 clients and $\lambda = 0.5$. Baselines are FedAvg, FedCS, Oort, TiFL, FedCor, FedGCS, and CriticalFL, as characterized in Table I. Oort and TiFL are simplified reimplementations that rank by local loss over estimated latency with a count-based bonus and cycle deterministically through the tiers, so neither carries Oort's pacer nor TiFL's probabilistic tier sampling. Their low coverage in Table II follows from that, and their accuracies are those of these variants.

A. Accuracy and Resource Cost

We estimate the client-training cost per round as

$$\text{TFLOPs}_{total} = \sum_{t=1}^{T} \sum_{i \in \mathcal{S}_t} E\, n_i\, C_{model} \times 10^{-12}, \tag{12}$$

where C_{model} is the per-sample forward-plus-backward cost in FLOPs, so one ResNet18 client-round on CIFAR-10 is about 4.13 TFLOPs. Energy accumulates tier power over it,

$$\mathcal{W}_{total} = \sum_{t=1}^{T} \sum_{i \in \mathcal{S}_t} \frac{\mathcal{P}_i\, T_{i,t}}{3600}, \tag{13}$$

where $\mathcal{P}_i$ is the device-tier power in watts and $T_{i,t}$ is in seconds, so $\mathcal{W}_{total}$ accumulates in watt-hours (Wh).

Since C_{model} is fixed within a dataset, cumulative TFLOPs is proportional to the samples the selected clients train on, so the mean shard per selection falls from 600/500/500 under FedAvg to 523/393/492. Energy adds the tier power per unit throughput, which MAML-Select brings to 0.967 of the pool average on CIFAR-100. These two factors account for the reported reductions, 12.9% and 4.1% on Fashion-MNIST, 21.5% and 4.2% on CIFAR-10, and 1.7% and 3.0% on CIFAR-100. The sample budget alone does not set the accuracy price, since FedCS trains on 46% fewer Fashion-MNIST samples and gives up 6.1 points where MAML-Select trains on 12.9% fewer and gives up 0.1.

Table II summarises all methods. MAML-Select matches FedAvg and FedGCS on Fashion-MNIST and matches FedGCS on CIFAR-100, where it exceeds FedCS, Oort, TiFL, FedCor and CriticalFL. On CIFAR-10 it gives up 3.8 pp against FedAvg for substantially lower cost. Its meta-update overhead of about 10^{-6} TFLOPs per round is negligible next to the client-training cost. CIFAR-100 is the one dataset not favourable on every axis, since FedGCS leads on accuracy, F1 and TFLOPs while MAML-Select leads on modelled energy and latency. Since (3) penalizes round latency, the achieved value is reported. The mean CIFAR-100 round latency is 2246 s for MAML-Select against 2624 s for FedAvg and 2538 s for FedGCS, and 50% accuracy arrives after 48.6 simulated hours against 56.1 for FedGCS and 47.7 for FedAvg.

Paired t-tests over the three seeds support the resource reductions against FedAvg on every dataset, with TFLOPs p-values of 0.009, 0.002 and 0.012 and energy p-values of 0.007, 0.001 and 0.003. Three seeds give a test little power,

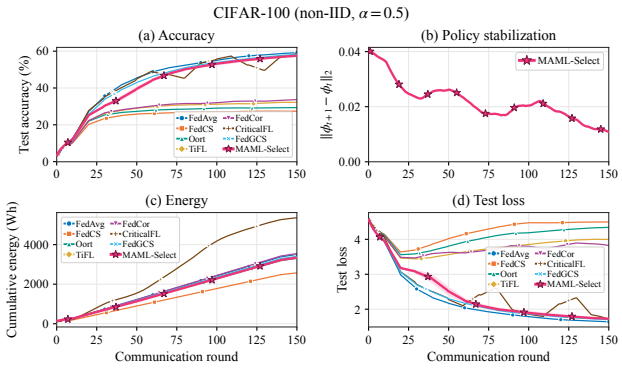

Fig. 2. CIFAR-100 convergence over 150 rounds. The panels report (a) accuracy, (b) the selector meta-update magnitude $\|\phi_{t+1} - \phi_t\|_2$ for MAML-Select , which shrinks and then settles at a small non-zero level, the behaviour Corollary 1 predicts under a drifting query objective, (c) cumulative modelled energy consumption, and (d) test loss.

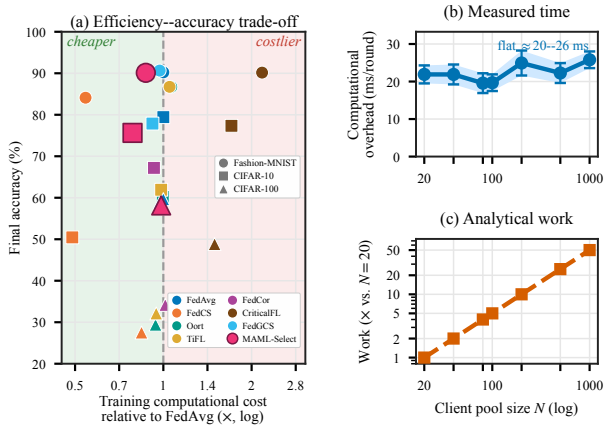

Fig. 3. (a) Efficiency and accuracy trade-off, where the x-axis is cumulative training cost relative to FedAvg on a log scale and $x < 1$ is cheaper, the y-axis is final accuracy, and marker shape encodes the dataset. (b) Measured per-round selection overhead, as a mean with 95% confidence intervals over 10 rounds. (c) The operation count $C(N, K)$ of (11) with $P = 4{,}673$, normalized to $N = 20$, both varying N at a 10% selection rate.

so a large p-value means a difference was not detected and is not evidence of equivalence. No accuracy difference against FedGCS is detected on CIFAR-10 or CIFAR-100, and the CIFAR-100 interval still admits a gap of about 1.8 pp either way. None is detected against FedAvg on Fashion-MNIST, whereas CIFAR-10 at -3.8 pp and CIFAR-100 at -1.5 pp trade accuracy for the lowest modelled energy in the table. Lowering λ from 0.5 to 0.1 raises CIFAR-10 accuracy under the protocol of Table III by about one point, so the gap reflects the harder ResNet18 task and not λ.

Fig. 2 summarises the CIFAR-100 behaviour, where MAML-Select stays close to FedAvg and FedGCS on accuracy and loss while remaining in the low-energy cluster, and panel (b) shows the meta-update settling.

Fig. 3(a) shows the efficiency and accuracy trade-off. Relative to FedAvg over shared seeds, MAML-Select lowers cumulative TFLOPs by 12.9%, 21.5% and 1.7% and modelled energy by 16.4%, 24.8% and 4.7% on the three datasets, for accuracy changes of -0.1, -3.8 and -1.5 percentage points, so the gain is resource efficiency at competitive accuracy.

Fairness uses Jain's index on the selection counts $\{p_i\}_{i=1}^{N}$, $J = \left(\sum_i p_i\right)^2 / \left(N \sum_i p_i^2\right)$, which lies in $[1/N, 1]$ and equals 1 for even selection. Coverage $\mathrm{Cov} = \frac{100}{N} \left|\{\, i : \sum_t a_{i,t} \geq 1 \,\}\right|$ is the percentage selected at least once. MAML-Select reaches 100% coverage on every dataset, with a Jain index of 0.61 to 0.78. The two differ. Coverage is guaranteed by the exploration slot (Proposition 2) whatever the learned ranking, whereas the Jain index measures how evenly the remaining $K - \varepsilon$ places are spread, and Sec. V-B shows the lower index reflects a preference for fast devices.

B. *Tier-Level Participation*

Coverage says how evenly clients are used but not which ones, so we also measured the share of selections per device tier on CIFAR-100 against a pool composition of 0.20/0.50/0.30. FedCS collapses onto the fast tier, placing all of its selections in Tier 3 and reaching only 10% coverage. FedAvg tracks the pool at 0.196/0.495/0.309 and FedGCS at 0.217/0.472/0.311, both with full coverage. MAML-Select shifts toward the fast tier at 0.116/0.446/0.437, yet keeps the slow tier represented at full coverage. What matters is the distinction between preferring fast devices and excluding slow ones, and only the former is compatible with Proposition 2.

C. *Coverage Without the Forced Cold Start*

The cold start visits every client once in the first 10 of 200 rounds, so the coverage figures could be an artifact of that schedule. Recomputing both metrics over the policy-driven rounds alone, across all 74 distinct MAML-Select runs and every λ, inner-step, width and feature-ablation setting, coverage is 100% in every individual run. Pooled over those runs Jain's index falls only slightly, from 0.755 to 0.737, 0.411 to 0.367, and 0.802 to 0.785. Pooling averages over settings, including large λ, so these sit below the default values of Table II.

Proposition 2 (Coverage from the exploration slot alone)**.** *Suppose every client is available each round and $\varepsilon \geq 1$. Then, after the cold start, every client is selected at least once in any window of $N - 1$ consecutive rounds, whatever the policy h_ϕ.*

Proof. Staleness obeys $\tau_{i,t+1} = 0$ if $i \in \mathcal{S}_t$ and $\tau_{i,t+1} = \tau_{i,t} + 1$ otherwise, and the slot goes to a client of maximal staleness. Fix j and a window starting at t, and suppose j is never selected in it. For j to miss the slot at round $s \geq t$, some $i_s \neq j$ with $\tau_{i_s,s} \geq \tau_{j,s}$ takes it and resets to 0. Since i_s resets at s, its staleness at $s' > s$ is $s' - s - 1$, so blocking j again would need $s' - s - 1 \geq \tau_{j,t} + (s' - t)$, that is $t - s \geq \tau_{j,t} + 1 \geq 1$, which is impossible for $s \geq t$. Each of the other $N - 1$ clients blocks j at most once, so j is selected within $N - 1$ rounds. Selection through the learned ranking also resets τ_j, which can only shorten the window.

With $N = 100$ and $T = 200$ the condition is met twice over, so coverage holds in every run. The cold start seeds the feedback buffer but does not create the result, and Jain's index depends mildly on it, so we report both.

TABLE II
BENCHMARK SUMMARY ACROSS NON-IID DATASETS. ACCURACY, F1, AND COVERAGE ARE IN PERCENT, AND ENERGY IS IN WATT-HOUR (WH). VALUES ARE THE MEAN ± STANDARD DEVIATION OVER SEEDS.

Method	Acc.	F1	TFLOPs	Energy	Jain	Cov.
Fashion-MNIST						
FedAvg	90.22±0.58	90.22±0.51	140±1	5752±72	0.95±0.00	100±0
FedCS	84.11±1.82	83.91±1.98	76±7	2772±211	0.10±0.00	10±0
Oort	86.70±0.19	86.42±0.26	149±8	5963±456	0.10±0.00	10±0
TiFL	86.73±0.34	86.51±0.34	147±6	6062±348	0.11±0.00	12±0
FedCor	89.35±0.50	89.22±0.57	121±5	4845±204	0.26±0.04	49±8
CriticalFL	90.16±0.47	90.11±0.42	304±143	12552±5860	0.98±0.01	100±0
FedGCS	90.65±0.29	90.63±0.26	136±4	5595±113	0.95±0.02	100±0
MAML-Select	**90.11±0.47**	**90.04±0.30**	**122±2**	**4809±79**	**0.77±0.06**	**100±0**
CIFAR-10						
FedAvg	79.43±1.84	79.36±1.78	8341±52	4798±24	0.95±0.00	100±0
FedCS	50.45±2.57	49.87±2.33	4081±600	2073±262	0.10±0.00	10±0
Oort	60.19±5.18	59.69±4.61	8303±1923	4677±856	0.10±0.00	10±0
TiFL	61.93±5.04	61.36±4.60	8200±2143	4800±1203	0.11±0.00	12±0
FedCor	67.20±5.33	66.81±5.37	7748±695	4452±435	0.20±0.02	31±5
CriticalFL	77.34±5.21	77.46±5.21	14239±4864	8205±2807	0.98±0.01	100±0
FedGCS	77.88±1.75	77.69±1.54	7656±278	4428±154	0.90±0.03	100±0
MAML-Select	**75.63±1.43**	**75.25±1.34**	**6549±118**	**3610±60**	**0.61±0.01**	**100±0**
CIFAR-100						
FedAvg	59.66±0.29	59.13±0.40	6331±6	3626±18	0.94±0.00	100±0
FedCS	27.52±0.29	26.39±0.09	5334±23	2659±12	0.10±0.00	10±0
Oort	29.41±1.73	28.13±1.73	5955±189	3421±181	0.10±0.00	10±0
TiFL	28.52±5.08	27.22±5.37	5961±35	3480±27	0.11±0.00	12±0
FedCor	30.86±4.70	29.39±5.00	6473±64	3663±8	0.12±0.00	14±1
CriticalFL	45.37±6.13	44.29±6.75	10235±1354	5879±779	0.98±0.01	100±0
FedGCS	58.66±0.11	58.08±0.20	6140±24	3526±4	0.82±0.02	100±0
MAML-Select	**58.15±0.51**	**57.66±0.33**	**6224±20**	**3457±6**	**0.78±0.03**	**100±0**

D. *Selector Convergence and Objective Drift*

We instrument a full 199-round run on each dataset, recording the inner-step change, the realized query objective, and the meta-update magnitude. Lemma 1 is confirmed exactly, since the inner step is non-increasing on the support loss in 198 of 198 adapted rounds on all three datasets, so $\beta = 10^{-2}$ lies inside the admissible range in practice.

The query objective is not monotone on any dataset. On Fashion-MNIST it falls from 3.312 to 0.015 with round-to-round jumps as large as 2.672, on CIFAR-10 it moves within $[0.015, 1.023]$ with jumps to 0.703, and on CIFAR-100 within $[0.055, 3.704]$ with jumps to 3.612. A fixed objective descended with $\eta \leq 1/L_q$ cannot produce increases of this size, so the sequence is direct evidence that q_t changes between rounds, which is the drift term V_T of Corollary 1. Logging both objectives at a common iterate measures it directly. Over 198 rounds it is -2.89 on Fashion-MNIST, 0.51 on CIFAR-10, and 1.79 on CIFAR-100, so $|V_T|/T$ is at most 0.015. The partial sums flatten instead of growing with the horizon, which places all three runs in the $V_T = o(T)$ regime of Remark 1, and Fashion-MNIST is negative and so falls in the benign case that remark identifies. These are single instrumented runs, so they measure the drift term without characterizing its distribution.

E. *Sensitivity and Ablation*

We probe the cost trade-off λ, the inner-loop steps, the selector width, and the six-feature state, each on the dataset where it is most informative. Fig. 4 reports the λ sweep. A larger λ weights cheaper clients more, so cost and energy fall on both datasets. Accuracy stays within about one point on Fashion-MNIST, whereas on CIFAR-10 the trade-off is sharper, falling from 64.9% at $\lambda = 0.1$ to 56.3% at $\lambda = 5$

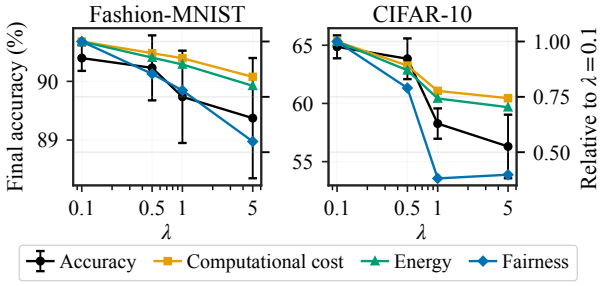

Fig. 4. Sensitivity to the cost trade-off λ on Fashion-MNIST (left) and CIFAR-10 (right). The black curve is accuracy, and cost, energy and fairness are normalized to their $\lambda = 0.1$ values. The CIFAR-10 sweep uses the 100-round protocol of Table III.

while cost drops by about 26% and energy by about 30%. Fairness also falls at large λ, so we keep $\lambda = 0.5$.

Table III depicts the choices of two ablations. On CIFAR-10, a single inner step gives the highest accuracy (63.8% at 100 rounds) and the lowest overhead (38.7 versus 45.2 ms at five steps), so we use one step. On Fashion-MNIST, accuracy stays within about 0.6 points as h grows over $\{16, 32, 64, 128\}$, or 401 to 17,537 weights, so $h = 64$ is robust.

The last group of Table III removes one state feature at a time. Accuracy changes stay within about 0.35 points, so the policy is robust to any single removal. The largest drop comes from the battery feature (-0.33 pp), with latency and loss also contributing. Removing the frequency feature raises accuracy slightly but lowers the Jain index the most, from 0.78 to 0.65, so the participation features chiefly protect fairness.

F. *Behaviour Under Severe Heterogeneity*

The results above use a Dirichlet concentration of $\alpha = 0.5$. Repeating the comparison at $\alpha = 0.1$ over three seeds, under

TABLE III

DESIGN ABLATIONS, GROUPED BY THE CHOICE BEING VARIED. ACCURACY IS THE MEAN ± STANDARD DEVIATION OVER THREE SEEDS. COST IS SELECTION OVERHEAD IN MILLISECONDS FOR THE FIRST TWO GROUPS AND CUMULATIVE TFLOPS FOR THE THIRD, AND A BLANK MARKS A QUANTITY NOT RECORDED. EACH GROUP IS A SEPARATE RUN SET FROM TABLE II AND IS READ WITHIN ITSELF, AND THE CIFAR-10 GROUP RUNS FOR 100 ROUNDS RATHER THAN 200. DEFAULTS ARE IN BOLD.

Setting	Acc. (%)	Cost	Jain
Inner-loop steps, CIFAR-10, cost in ms			
1	**63.8±1.8**	**38.7**	
2	63.1±1.8	39.8	
5	62.2±2.4	45.2	
Selector width h, Fashion-MNIST, cost in ms			
16	90.0±0.3	10.2	
32	90.6±0.3	10.4	
64	**90.1±0.5**	**10.2**	
128	90.3±0.7	11.6	
State features, Fashion-MNIST, 200 *rounds, cost in TFLOPs*			
Full state vector	**90.23±0.55**	**122**	**0.78**
w/o loss	90.16±0.66	123	0.79
w/o gradient norm	90.30±0.24	122	0.78
w/o latency	90.14±0.14	119	0.70
w/o battery	89.90±0.44	122	0.79
w/o frequency	90.47±0.13	115	0.65
w/o staleness	90.37±0.19	123	0.78

the protocol of Section V-A in all other respects, gives MAML-Select 55.8 ± 6.3 TFLOPs against 140.0 ± 1.3 for FedAvg on Fashion-MNIST and 4423 ± 581 against 8342 ± 89 on CIFAR-10. These are reductions of 60% and 47%, at $p = 0.002$ and $p = 0.007$ on a paired test. Accuracy falls, to 83.4 ± 4.3 against 86.6 ± 2.7 on Fashion-MNIST and 41.2 ± 6.2 against 58.6 ± 7.7 on CIFAR-10, with FedGCS between the two at 85.1 ± 2.2 and 50.5 ± 16.9. The cost term of (7) concentrates selection on fast devices, and at $\alpha = 0.1$ those devices hold a narrower slice of the label space, so the cohort sees fewer classes per round. This is the operating boundary of the method. MAML-Select suits moderate heterogeneity, and at $\alpha = 0.1$ the compute saving carries an accuracy cost on CIFAR-10.

VI. CONCLUSION AND FUTURE WORK

We introduced MAML-Select, an online adaptive client-selection framework for heterogeneous FL that learns a lightweight ranking policy over learning feedback and system state, and adapts it before each selection decision. Across Fashion-MNIST, CIFAR-10, and CIFAR-100 it holds competitive accuracy while lowering cumulative computational cost, modelled energy, and round latency. The λ study shows a mild trade-off on Fashion-MNIST and a sharper one on CIFAR-10, and the ablations confirm that a single inner step, a compact policy network, and the full six-feature state suffice. The evaluation covers two heterogeneity levels, one cohort size and three vision datasets, and a no-adaptation control is left open. Section V-F marks the boundary at $\alpha = 0.1$. Future work will broaden the evaluation across pool sizes and naturally federated benchmarks, and will study an asynchronous extension together with a differentially private client-state vector so that selection does not leak device information.

REFERENCES

[1] B. McMahan *et al.*, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*. PMLR, 2017, pp. 1273–1282.

[2] S. P. Karimireddy *et al.*, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. ICML*. PMLR, 2020, pp. 5132–5143.

[3] V. S. Mai, R. J. La, and T. Zhang, "A study of enhancing federated learning on non-IID data with server learning," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 11, pp. 5589–5604, 2024.

[4] R. Tapwal, "Stragglers reimagined: Explainability-driven adaptive federated learning for resource constrained IoMT system," *IEEE Transactions on Artificial Intelligence*, vol. 7, no. 2, pp. 1002–1011, 2026.

[5] A. Forootani and R. Iervolino, "Asynchronous federated learning with nonconvex client objective functions and heterogeneous dataset," *IEEE Transactions on Artificial Intelligence*, vol. 7, no. 5, pp. 2610–2624, 2026.

[6] K. Borazjani *et al.*, "Redefining non-IID data in federated learning for computer vision tasks," *IEEE Transactions on Artificial Intelligence*, 2026, early access.

[7] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *Proc. IEEE ICC*. IEEE, 2019, pp. 1–7.

[8] Z. Chai *et al.*, "TiFL: A tier-based federated learning system," in *Proc. ACM HPDC*, 2020, pp. 125–136.

[9] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *Proc. USENIX OSDI*, 2021, pp. 19–35.

[10] M. Tang *et al.*, "FedCor: Correlation-based active client selection strategy for heterogeneous federated learning," in *Proc. IEEE/CVF CVPR*. IEEE, 2022, pp. 10 102–10 111.

[11] Y. Xu *et al.*, "Federated learning with client selection and gradient compression in heterogeneous edge systems," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5446–5461, 2024.

[12] G. Yan, H. Wang, X. Yuan, and J. Li, "CriticalFL: A critical learning periods augmented client selection framework for efficient federated learning," in *Proc. ACM SIGKDD*, 2023, pp. 5034–5044.

[13] Z. Ning *et al.*, "FedGCS: A generative framework for efficient client selection in federated learning via gradient-based optimization," in *Proceedings of the 33rd International Joint Conference on Artificial Intelligence*, 2024, pp. 4760–4768.

[14] H. Wang, Z. Kaplan, D. Niu, and B. Li, "Optimizing federated learning on non-IID data with reinforcement learning," in *IEEE INFOCOM*. IEEE, 2020, pp. 1698–1707.

[15] Q. Pan, H. Cao, Y. Zhu, J. Liu, and B. Li, "Contextual client selection for efficient federated learning over edge devices," *IEEE Transactions on Mobile Computing*, vol. 23, no. 6, pp. 6538–6548, 2024.

[16] K. Kazemi, M. H. Badiei, and H. Kebriaei, "Robust peer-to-peer federated learning with deep reinforcement learning based client selection against data poisoning attacks," *IEEE Transactions on Artificial Intelligence*, vol. 7, no. 1, pp. 262–271, 2026.

[17] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. ICML*. PMLR, 2017, pp. 1126–1135.

[18] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach," *Proc. NeurIPS*, vol. 33, pp. 3557–3568, 2020.

[19] H. A. Tran, C. Ta, and T. X. Tran, "FedImp: Enhancing federated learning convergence with impurity-based weighting," *IEEE Transactions on Artificial Intelligence*, vol. 7, no. 3, pp. 1652–1665, 2026.

[20] S. Arisdakessian, O. A. Wahab, A. Mourad, and H. Otrok, "Towards vox populi in federated learning: A fair and inclusive client selection framework," *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 1997–2011, 2026.

[21] X. Wu, R. He, Z. Sun, and T. Tan, "A light CNN for deep face representation with noisy labels," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 11, pp. 2884–2896, 2018.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*. IEEE, 2016, pp. 770–778.

[23] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.